20 | Interacting with the device

Resources > Chapter 20 > Taking photos

20.1 Taking a picture

It wouldn't be a surprise if you were asked to take a photo with your app and eventually store the image file you've acquired. The official *camera* [1] package allows you to work with any camera of your device, display previews and acquire photos.

[1] https://pub.dev/packages/camera

The app we're going to build shows a **live** preview of what's going to be captured and an icon at the bottom which stores the picture in the temporary directory. There are some packages you need to import:

- *camera*, to work with the cameras of the device;
- *path_provider*, to easily find folders' paths;
- *path*, to easily get paths on any platform.

Please note that in order to successfully compile the *camera* plugin you have to change the minimum Android sdk version to 21 at `app/build.gradle`. Look for the `minSdkVersion` label and set it to be 21 (the default value might be lower, such as 16). For iOS users instead add two new entries in `ios/Runner/Info.plist`

- Privacy - Camera Usage Description
- Privacy - Microphone Usage Description

Associate to each key a meaningful description that will be prompted to the user. At this point the setup is ready and we can start creating the app.

1. In the initialization phase, we need to retrieve the list of all available cameras of the device by using `availableCameras()`. Generally there are two, one at the front and one at the back.

   ```
   void main() async {
       // This call makes sure the camera plugin has been
       // properly initialized and it's ready to be used.
       WidgetsFlutterBinding.ensureInitialized();

       final cameras = await availableCameras();
       final firstCamera = cameras.first;

       runApp(...);
   }
   ```

   The `cameras` variable is of type `List<CameraDescription>` so calling `cameras.first` is equivalent to writing `cameras[0]`.

2. In order to establish a connection with the device's camera you need a `CameraController` and, like any other controller, it has to be initialized and disposed. For this reason, create a `StatefulWidget` which will take care of the controller's lifecycle.

```
class TakePicture extends StatefulWidget {
    final CameraDescription camera;
    TakePicture({required this.camera});

    @override
    _TakePictureState createState() => _TakePictureState();
}

class _TakePictureState extends State<TakePicture> {
    late final CameraController _controller;
    late final Future<void> _initController;

    @override
    void initState() {
        super.initState();
        _controller = CameraController(
            widget.camera,
            ResolutionPreset.high,
        );

        _initController = _controller.initialize();
    }

    @override
    void dispose() {
        _controller.dispose();
        super.dispose();
    }

    // build...
}
```

You'll see that `_initController` is used in a `FutureBuilder<T>` to wait for the loading of the camera preview screen. You can decide to take high or low quality pictures with one of these controller's constants:

- `ResolutionPreset.low` (240p),
- `ResolutionPreset.medium` (480p),

- `ResolutionPreset.high` (720p),
- `ResolutionPreset.veryHigh` (1080p),
- `ResolutionPreset.ultraHigh` (2160p)

Alternatively, if you don't know which is the maximum resolution supported by the device, you can use `ResolutionPreset.max` which automatically picks the highest preset possible. For the sake of the example we've simply used a `Column` but you could try to make a more complex layout.

```
@override
Widget build(BuildContext context) {
    return Column(
        mainAxisAlignment: MainAxisAlignment.spaceAround,
        children: <Widget>[
            FutureBuilder<void>(...),

            IconButton(...)
        ],
    );
}
```

3. The `FutureBuilder<void>` widget is going to `await` for the initialization of the camera because, as soon as it's ready, we want it to give a live preview of what it's framing.

```
FutureBuilder<void>(
    future: _initController,
    builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.done) {
            return Expanded(
                child: Center(
                    child: AspectRatio(
                        aspectRatio: _controller.value.aspectRatio,
                        child: CameraPreview(_controller),
                    ),
                ),
            );
        }

        return const Center(
```

```
            child: CircularProgressIndicator(),
          );
        },
      ),
```

We recommend the usage of the `AspectRatio` widget as it shows the preview with proper proportions between width and height. It's not compulsory, you could have put the `CameraPreview` widget directly inside `Expanded` and it would have covered the entire available space.

```
return Expanded(
    child: CameraPreview(_controller),
)
```

Since we're inside a `Column` we need somehow to constrain the height and `Expanded` does exactly this. Alternatively you could have used a `Container` or a `SizedBox`.

4. In the `FutureBuilder<void>` we've placed the button to actually take the photo and store the image in the temporary directory with a "random" filename.

```
IconButton(
    icon: const Icon(Icons.photo_camera),
    onPressed: () => _takePhoto(context),
)
```

We underline again the fact that it's a good idea putting the business logic outside of the UI logic and thus the body of the `onPressed` callback has been created in a separated function.

```
void _takePhoto(BuildContext context) async {
    // Ensure the controller is ready
    await _initController;

    // File name and path
    final dir = await getTemporaryDirectory();
    final name = "mypic_${DateTime.now()}.png";

    // Store the picture at the given location
    final fullPath = path.join(dir.path, name);
    await _controller.takePicture(fullPath);

    Scaffold.of(context).showSnackBar(
```

```
            SnackBar(
              content: const Text("Picture taken!"),
              duration: const Duration(milliseconds: 600),
            )
        );
}
```

A common strategy to avoid file name conflicts is relying on the current date and time in the name. While this approach is not bullet proof, for simple tasks it's good enough. Note the usage of *join*:

```
final fullPath = path.join(dir.path, name);
```

Combining paths in cross-platform apps is not easy because you have to take into account the filesystem structure of every single supported OS. This is something low-level that you don't want to deal with, so Flutter has a package ready for you and it's called *path*!

Resources > Chapter 20 > Sensors

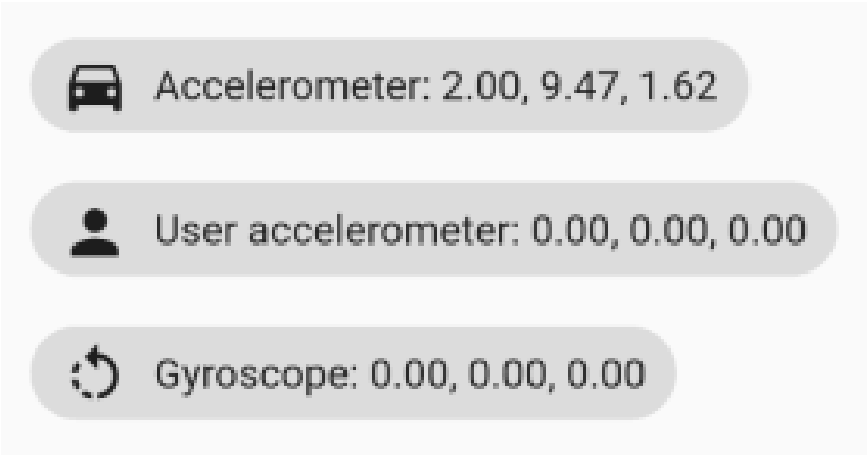
20.2 Working with sensors

Nowadays almost any device has built-in sensors to measure orientation, motion or other environmental conditions with a very high accuracy. To monitor the three-dimensional data the device is able to provide you can use the official *sensor* [2] package.

- `GyroscopeEvents`: data about the rotation of the device;
- `AccelerometerEvent`: these data let you know in which direction the device is moving, keeping into account the gravity;
- `UserAccelerometerEvent`: similar to `AccelerometerEvent` but it doesn't care about the gravity

We're creating a very basic app that shows data obtained from the sensors at the center of the screen. They are useful when you want to implement motion-based functionalities in your app.

[2] https://pub.dev/packages/sensors



The API of the sensor package is very simple to use and it relies on streams.

1. The `Wrap` widget places the children side by side in the horizontal or vertical direction. It's very similar to `Row` and `Column` but with the difference that its sizes are **constrained**: when there is no more space to lay out the children, it wraps to the next column/row.

```
class SensorWidget extends StatelessWidget {
    const SensorWidget();

    @override
    Widget build(BuildContext context) {
        return Wrap(
            spacing: 0.9,
            direction: Axis.vertical,
            children: const <Widget>[
                AccelerometerData(),
                UserAccelerometerData(),
                GyroscopeData()
            ],
        );
    }
}
```

Here we're placing a series of widgets vertically with a certain space in between. If you try to rotate the screen and set a big gap, such as `spacing: 150`, you'll see that widgets are rearranged differently and no overflow errors appear.

2. As you know, the `StreamBuilder<T>` widget deals with streams and we strongly recommend you to **always** supply some initial data to it. This ensures that the first frame will show useful data rather than `null`, which is safe.

```
class AccelerometerData extends StatelessWidget {
    static final initialData = AccelerometerEvent(0, 0, 0);
    const AccelerometerData();

    @override
    Widget build(BuildContext context) {
        return StreamBuilder<AccelerometerEvent>(
            stream: accelerometerEvents,
            initialData: initialData,
            builder: (context, accel) {
                final data = accel.data;

                if (data != null) {
                    final x = data.x.toStringAsFixed(2);
                    final y = data.y.toStringAsFixed(2);
                    final z = data.z.toStringAsFixed(2);

                    return ChipWithIcon(
                        icon: Icon(Icons.drive_eta),
                        text: "Accelerometer: $x, $y, $z"
                    );
                }

                return const Text("No data to display.");
            },
        );
    }
}
```

The `accelerometerEvents` stream is provided by the *sensor* package and it continuously sends a series of three-dimensional data we're showing on the UI. At the center of the screen

there's a material `Chip`, a rounded box representing short spans of text.

```
class ChipWithIcon extends StatelessWidget {
    final Icon icon;
    final String text;
    const ChipWithIcon({
        required this.icon,
        required this.text
    });

    @override
    Widget build(BuildContext context) {
        return Chip(
            avatar: icon,
            padding: const EdgeInsets.fromLTRB(10, 5, 10, 5),
            labelPadding: const EdgeInsets.only(left: 10),
            label: Text(text),
        );
    }
}
```

We haven't discussed the implementations of the other two listeners because they're basically identical to `AccelerometerData` with the only difference that they subscribe to a different stream.

Resources > Chapter 20 > Geolocation

20.3 Working with Geolocation

A very popular geolocation package for Flutter is *geolocator* [3], well-known for its easy way to access the device's location services. Before starting with the examples, apply the following changes to your project:

- On **Android** you need to add two permissions in the manifest at `android/app/src/main`.

```
<uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION" />
```

[3] https://pub.dev/packages/geolocator

```
<uses-permission
    android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

In addition, make sure the `gradle.properties` file has AndroidX enabled by checking if these two lines are present. They are required for version 3.0.0 and above of the plugin to work.

```
android.useAndroidX=true
android.enableJetifier=true
```

- On **iOS** you have to setup the permission in the `Info.plist` file.

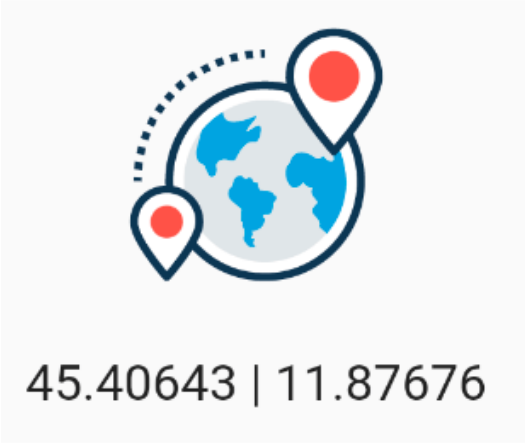
```
<key>NSLocationWhenInUseUsageDescription</key>
<string>Give me location permissions pls</string>
```

Note that in early versions you had to access members using `class Geolocator` which implemented the *singleton* design pattern via `factory` constructor. Now the class doesn't exist anymore: the package just exposes a series of top-level functions.

```
var position = await getCurrentPosition(
    desiredAccuracy: LocationAccuracy.high
);

var lat = position.latitude;
var lon = position.longitude;
var alt = position.altitude;
```

You could also use `position.speed` to indicate the speed at which the device is moving but it's not always available. There's also the possibility to listen for position changes thanks to `getPositionStream()` which returns a `Stream<Position>`.



In this very simple example we have an SVG image at the top and below there is a string reporting the current latitude and longitude. If we went out for a walk, we would see the text automatically updating to reflect the new coordinates thanks to the stream.

1. The setup for the `GPSWidget` is very easy to understand. We have decided to make the `_stream` variable `static` to be able to declare a `const` constructor.

```
class GPSWidget extends StatelessWidget {
    static final _stream = getPositionStream();
    const GPSWidget();

    @override
    Widget build(BuildContext context) {
        ...
    }

}
```

2. As usual, we're going to use the `StreamBuilder<T>` widget to subscribe to the stream and constantly update the text below the image.

```
return Wrap(
    spacing: 20,
    direction: Axis.vertical,
    crossAxisAlignment: WrapCrossAlignment.center,
    children: <Widget>[
```

```
            SvgPicture.asset(
                "assets/geolocate.svg",
                height: 70,
            ),

            StreamBuilder<Position>(
                stream: _stream,
                builder: (context, positionData) {
                    if (positionData.hasData) {
                        final data = positionData.data;

                        if (data != null) {
                            final lat = data.latitude.toStringAsFixed(5);
                            final lon = data.longitude.toStringAsFixed(5);

                            return Text("$lat | $lon");
                        } else {
                            return const Text("No data available.");
                        }
                    }

                    return const CircularProgressIndicator();
                },
            )
        ],
    );
```

Alternatively to `Wrap` you could have achieved the same result with a column but it would have required more boilerplate code. It has no `spacing` property so you'd have to deal with `Padding` or another "spacer" widget.

```
// A 'SizedBox' can be good to add spaces between widgets
const SizedBox(
    height: 20
);
```

If you wish to calculate the distance between two points in the world, use `distanceBetween()` and pass latitude and longitude of the starting and ending point. The distance is expressed in meters.

```
final double distance = await distanceBetween(
    52.2165157, 6.9437819, // starting point lat-long
    52.3546274, 4.8285838  // ending point lat-long
);
```

Another very interesting package is called *geocoding* because it gives you the possibility to translate latitude/longitude coordinates into an address and vice versa. It's very easy to use:

```
final placemarks = await placemarkFromCoordinates(41.909986, 12.3959135);

if (placemarks.isNotEmpty) {
    final result = placemarks[0];
    debugPrint("${result.administrativeArea}");  // Lazio
    debugPrint("${result.locality}");            // Rome
    debugPrint("${result.country}");             // Italy
}
```

Ideally the `placemarks` list will contain only 1 item that exactly matches what you're looking for but in certain cases there might be multiple results. It's of type `List<Placemark>` and it has a ton of properties [4] (in addition to the ones we've used above).

```
var locations = await locationFromAddress("Rome");

if (locations.isNotEmpty) {
    var result = locations[0];
    debugPrint("${result.latitude}");  // 41.9...
    debugPrint("${result.longitude}"); // 12.3...
}
```

This is very similar to the previous example but instead of passing the latitude and the longitude, we pass the name of the desired location.

20.4 Platform-specific packages

Flutter's official pub [5] page has many packages to interact with some platform-specific APIs. Be sure to also have a look at the GitHub repo [6] for a summary table that redirects you to the install page of the packages.

[4] https://pub.dev/packages/geocoding
[5] https://pub.dev/flutter/packages
[6] https://github.com/flutter/plugins

20.4.1 Battery level

The *battery* package lets you extract various information about your device's battery such as the current charge level. Once it's been imported you just need to create an instance of `class Battery` and then use its asynchronous API.

```
Future<int> level() async {
    var b = Battery();
    return await b.batteryLevel;
}
```

If you wanted your device to listen for battery changes, subscribe to the `onBatteryStateChanged` stream which exposes a `BatteryState`. It is an `enum` representing the current state of the battery.

```
enum BatteryState {
    /// The battery is completely full of energy.
    full,

    /// The battery is currently storing energy.
    charging,

    /// The battery is currently losing energy.
    discharging
}
```

20.4.2 Device info

To get a lot of info about the device in which your app is running you can use the *device_info* [7] package from the official Flutter team. It's just a matter of instantiating `DeviceInfoPlugin()` and then using the getters.

```
final deviceInfo = await DeviceInfoPlugin().androidInfo;
//final deviceInfo = await DeviceInfoPlugin().iosInfo;

debugPrint(deviceInfo.display);
debugPrint(deviceInfo.brand);
debugPrint(deviceInfo.product);
```

[7] https://pub.dev/packages/device_info

```
debugPrint(deviceInfo.manufacturer);
// and much more...
```

There is a very long series of data this package is able to extract so be sure to check out the documentation.

Resources > Chapter 20 > Connectivity

20.4.3 Internet connectivity

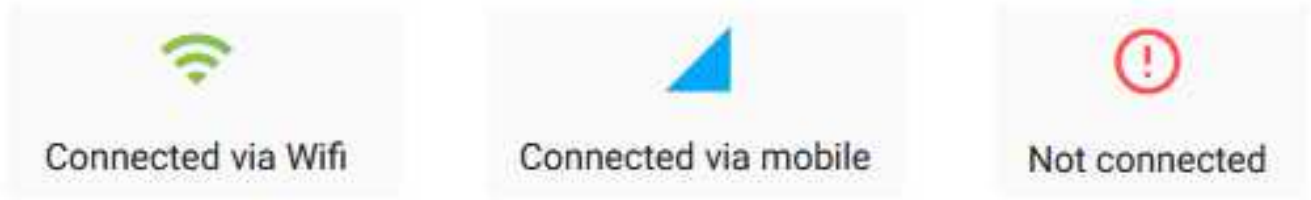
So far we've used *dio* and *http* to make HTTP requests, download/upload files and handle potential errors. They're not able to detect the connection status before making the actual request but the *connectivity* [8] package can.

```
var status = await Connectivity().checkConnectivity();

switch (status) {
    case ConnectivityResult.wifi:
        debugPrint("Connected via wifi");
        break;
    case ConnectivityResult.mobile:
        debugPrint("Connected via mobile");
        break;
    case ConnectivityResult.none:
        debugPrint("not connected");
        break;
}
```

The `checkConnectivity()` method returns `enum ConnectivityResult` which lets us know if we are re connected to a network (via wifi or mobile) or not. In addition, we can also listen to connectivity changes while the app is running thanks to streams.

[8] https://pub.dev/packages/connectivity



Thanks to a `StreamBuilder<ConnectivityResult>` we are able to listen to changes and show proper widgets accordingly. You've seen this pattern many times up to now so it should be easy to understand.

1. The `Connectivity` class implements the singleton pattern so it will always return the same instance via `factory` constructor. For this reason, we can "cache" an instance in a `static` variable to define a constant constructor.

```
class ConnectionWidget extends StatelessWidget {
    static final _conn = Connectivity();
    const ConnectionWidget();

    @override
    Widget build(BuildContext context) {
        ...
    }
}
```

2. For each connection type (wifi, mobile or none) we're creating the widget representing the status with an icon and the text.

```
class WifiConnectionWidget extends StatelessWidget {
    const WifiConnectionWidget();

    @override
    Widget build(BuildContext context) {
        return Wrap(
            spacing: 20,
            crossAxisAlignment: WrapCrossAlignment.center,
            direction: Axis.vertical,
            children: const <Widget>[
```

```
                    Icon(Icons.wifi),
                    Text("Connected to WiFi")
                ],
            );
        }
    }

    class MobileConnectionWidget extends StatelessWidget { ... }

    class NoConnectionWidget extends StatelessWidget { ... }
```

3. At this point we can setup the `StreamBuilder<ConnectivityResult>` so that it can listen to connectivity changes and display the proper widget. A result of type `none` indicates that there's no internet connectivity at all.

```
@override
Widget build(BuildContext context) {
    return StreamBuilder<ConnectivityResult>(
        stream: _conn.onConnectivityChanged,
        builder: (context, status) {
            if (status.hasData) {
                final data = status.data;

                if (data != null) {
                    switch (data) {
                        case ConnectivityResult.wifi:
                            return const WifiConnectionWidget();
                        case ConnectivityResult.mobile:
                            return const MobileConnectionWidget();
                        case ConnectivityResult.none:
                            return const NoConnectionWidget();
                    }
                } else {
                    debugPrint("Whoops");
                }
            }

            return const CircularProgressIndicator();
        },
```

```
        );
    }
```

20.4.4 Shared preferences

The *shared_preferences* [9] package is very useful when you have a small collection of key-value data to save. Those data persist on the disk so even if you close the app, they will be available again at the next startup. Some scenarios in which you could use them are:

- storing some flags you app has to read at startup, such as styling configurations set by the user;
- storing some app's preferences such as the currently selected download directory;
- storing all those "small" data like numbers or strings you want to quickly read without having to setup a database.

Use `class SharedPreferences` when you want to work with `int`, `double`, `String` or `bool`. If you have a lot of configurations to store or a complex data structure, consider using a more reliable data structure such as a database [10].

```
final prefs = await SharedPreferences.getInstance();

// Store a value
await prefs.setInt('age', 23);

// Read a value
var value = prefs.getInt('age') ?? -1;
```

If `age` wasn't an integer an exception would occur. Do not store critical data using shared preferences nor rely on them to build you app's storage; they are just a simple way to store small data without having the overhead of managing a database.

```
SharedPreferences.setMockInitialValues({
    "name": "Robert",
    "age": 28
});
```

Use `setMockInitialValues()` in your tests to manually populate `SharedPreferences` with some initial values.

[9] https://pub.dev/packages/shared_preferences
[10] See appendix B.2